1 Brief description

A stochastic cellular automata running a 1k-RAM, 30kHz 6502 CPU pseudo-asynchronously at each cell.

2 Introduction

A cellular automata where each cell runs a virtual 6502, a classic CPU released in 1975. The appeal to retro computing is designed to win a wide audience, to support which we will also develop mobile/serverless app infrastructure to allow mobile code to run on peoples' phones and hop between phones, with mutation and selection. A client viewer layer will allow visual interpretation of virtual memory, as well as local policies e.g. migration to/from server (and/or local neighborhood) or suppression of particular programs. The long-term goal is to obtain a dataset that will allow us to ask whether we can use the tools of population genetics and phylogenetics to study mobile code on a large scale. Subprojects will include the design of various agent-based and cellular automata models from physics, biology, chemistry, and other fields of science and A-Life. To motivate broad adoption we will offer prizes for the most creative and most virally successful programs.

- The paged storage is addressed as a square array of cells with $B = 256$ cells per side and $M = 1024$ bytes per cell, so there are $B^2 = 65,536$ (64k) cells requiring $B^2M = 64$Mb of storage.

- The memory-mapped neighborhood is 7x7 cells around the local origin (0,0). Cell n occupies the space from $1024n$ bytes to $1024n + 1023$. The relationship between cell index, memory location, and offset (x, y) are shown in these tables

  Memory pages.

	$x=-3$	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$	$x=3$
$y=3$	C0..C3	B0..B3	90..93	64..67	74..77	94..97	B4..B7
$y=2$	AC..AF	60..63	50..53	24..27	34..37	54..57	98..9B
$y=1$	8C..8F	4C..4F	20..23	04..07	14..17	38..3B	78..7B
$y=0$	70..73	30..33	10..13	00..03	08..0B	28..2B	68..6B
$y=-1$	88..8B	48..4B	1C..1F	0C..0F	18..1B	3C..3F	7C..7F
$y=-2$	A8..AB	5C..5F	44..47	2C..2F	40..43	58..5B	9C..9F
$y=-3$	BC..BF	A4..A7	84..87	6C..6F	80..83	A0..A3	B8..BB

  Each time execution resumes after an interrupt, the origin is resampled and the neighborhood rotated by a random multiple of 90 degrees. Zero page locations 0xF0-0xF8 are designated as "oriented registers", whose top 6 bits are rotated along with the memory map when a different orientation is sampled. (Note: 0xF9, the storage location for PCHI, is also auto-rotated but is considered controller-reserved.)

  Cell numbers:

	$x=-3$	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$	$x=3$
$y=3$	48	44	36	25	29	37	45
$y=2$	43	24	20	9	13	21	38
$y=1$	35	19	8	1	5	14	30
$y=0$	28	12	4	0	2	10	26
$y=-1$	34	18	7	3	6	15	31
$y=-2$	42	23	17	11	16	22	39
$y=-3$	47	41	33	27	32	40	46

Cell coordinates:

Range	Directions
0	Origin
1..4	N=(0,+1), E=(+1,0), S, W
5..8	NE, SE, SW, NW
9..12	N^2, E^2, S^2, W^2
13..20	N^2E, NE^2, SE^2, S^2E, S^2W, SW^2, NW^2, N^2W
21..24	N^3, E^3, S^3, W^3
25..27	N^2E^2, S^2E^2, S^2W^2, N^2W^2
28..35	N^3E, NE^3, SE^3, S^3E, S^3W, SW^3, NW^3, N^3W
36..43	N^3E^2, N^2E^3, S^2E^3, S^3E^2, S^3W^2, S^2W^3, N^2W^3, N^3W^2
44..48	N^3E^3, S^3E^3, S^3W^3, N^3W^3

- The area from 0xE000-0xEE3F contains some read-only lookup tables for mapping various transformations inside the neighborhood (operations that yield vectors outside the neighborhood return 0xFF)

Table start S	Meaning of $S[i]$
0xE000 $+64j$	Vector addition $v_i + v_j$
0xEC40	Rotation 90° clockwise
0xEC80	Rotation 180°
0xECC0	Rotation 270°
0xED00	Reflection about x-axis
0xED40	Reflection about y-axis
0xED80	X coordinate of v_i plus 3
0xEDC0	Y coordinate of v_i plus 3
0xEE00	See below

  The table from 0xEE00-0xEE3F contains the mapping from (x, y) coordinates to cell indices, with y ascending fastest, starting from cell (-3,-3); so the byte in 0xEE00$+(y+3)+64(x+3)$ is the index of cell with relative offset (x, y) for $-3 \leq x, y \leq 3$.

- We aim to clock the entire board at the rate of a 1981 BBC Micro (2Mhz), so each cell updates at 30.5Hz. (This is a lower bound. There is no reason you can't run the board faster.)

- Interrupts arrive as an (approximately) Poisson process with an average rate of 1 per 4,096 cycles.

Interrupt model. Pre-emptive scheduling is conceptually an IRQ (which the 6502 program can mask using SEI/CLI) followed by an NMI (which it cannot). Memory writeback happens, if the program allows it, between the IRQ and the NMI. Setting the I flag (SEI) thus creates an atomic transaction: writes accumulate until a BRK commits them (software interrupt), and are reverted if a timer interrupt fires instead.

A timer interrupt is handled as follows:

  - If the interrupt disable flag (I) is set, all writes made since the last interrupt are reverted (atomic abort). Otherwise:
  - The B flag (bit 4) is cleared in P.
  - CPU registers are written to the last seven bytes of zero page (0xF9–0xFF).
  - The memory-mapped neighborhood is copied from RAM to storage (un-rotating oriented registers at 0xF0–0xF9).
  - A new origin cell (i, j) and orientation is randomly sampled. The memory-mapped neighborhood of that cell is loaded from storage to RAM.
  - The last four bytes of zero page are overwritten with pseudorandom numbers.
  - Optional (via the `hasCompass` hyperparameter): the scheduler writes the current orientation vector (left-shifted by two bits) to 0xFA, so programs can detect their orientation and adjust their behavior accordingly. If `hasCompass` is disabled, the scheduler writes zero to 0xFA instead.
  - CPU registers are restored from the last seven bytes of zero page (prior to their being overwritten).

- A BRK software interrupt costs 7 CPU cycles, matching the real NMOS 6502 (2 to fetch opcode + operand, 3 to push PC and P to stack, 2 to read the IRQ vector). Undocumented opcodes are treated as BRK 0 at the same 7-cycle cost. Key differences from timer interrupts:
  - The I flag is ignored: writes always commit after BRK (the "NMI" is unconditional).
  - The operand byte b (the byte following BRK) is examined *before* registers are saved. Copy and swap operations (below) use the cell's pre-interrupt state. This means a BRK copy produces a child that inherits the *previous* scheduling's saved registers, not the post-BRK state.
  - The B flag (bit 4) is set in the saved P, following the 6502 convention that BRK and PHP set B while IRQ and NMI clear it. This enables fork detection: after a BRK copy, the child inherits B=0 (pre-BRK state) while the parent gets B=1. Programs can read byte 0xFB and test bit 4.

– BRK with operand 0 resets PC to 0x0000 (preventing zero-filled cells from marching their PC into the RNG region). All other operands advance PC past the BRK+operand as usual.

– The operand byte b selects an operation. Swap operations exchange the origin cell (cell 0) with any of the 48 other cells in the 7×7 neighborhood. Noisy copy duplicates the origin cell to a neighbor through a noise gate.

Operand b	Operation
0	Resets PC to 0x0000. Yields to scheduler.
$1 \leq b \leq 48$	Swap cell 0 with cell b. Gated by `brkOps.swap.enabled`. Yields to scheduler.
$49 \leq b \leq 96$	Noisy copy: origin cell is copied to cell $b-48$ (cells 1–48), subject to the noise model (see below). Gated by `brkOps.copy.enabled`. Yields to scheduler.
97	Sync interrupt request: X and Y registers specify the period in cycles (X = low byte, Y = high byte). The next interrupt for this cell is scheduled at the nearest future absolute multiple of the period, computed from the global board time. Gated by `brkOps.sync.enabled`. Yields to scheduler.
98	Async interrupt request: X and Y registers specify the delay in cycles (X = low byte, Y = high byte). The next interrupt for this cell is scheduled after that many cycles from the current time. Gated by `brkOps.async.enabled`. Yields to scheduler.
99–255	Unassigned by default (no operation). Extensible via `brkOps` registry. Yields to scheduler.

If a BRK operand's operation is not enabled in `brkOps` (or is not registered at all), the BRK simply yields to the scheduler with no effect.

Cycle costs and `nSwapCycles`

BRK operations that manipulate cell data (swap and noisy copy) require the Shadow OS (Section 7) to perform work on the program's behalf. The `nSwapCycles` hyperparameter sets the minimum number of scheduler cycles that must remain before the next timer interrupt for the operation to succeed; if fewer cycles remain, the BRK simply yields with no effect.

Swap is an $O(1)$ operation: the Shadow OS remaps page-table entries in the paged storage hardware, exchanging the two cells without copying data. Noisy copy is $O(M)$: each of the $M = 1024$ bytes must be streamed through the noise gate. As a result, swap has a lower cycle cost than noisy copy. A reasonable default might be `nSwapCycles` $\approx$ 15,000 (slightly above the copy cost of $\sim$14,400

Shadow OS cycles), but the reference implementation defaults to 0 (no gating) for backwards compatibility.

Noisy copy model

The noisy copy operation (operands 49–96) copies all 1024 bytes of the origin cell to the destination. Each bit is independently resampled with probability ε (`pBitNoise`), and faithfully copied with probability $1 - \varepsilon$. When a bit is resampled, it becomes 0 with probability q (`pBitNoiseZero`) and 1 with probability $1 - q$. The default is $\varepsilon = 1/2048$ and $q = 0.5$ (fair coin), giving approximately 1 bit error per 256-byte page copied ($\sim$4 errors per cell). Setting $q = 1$ gives pure erasure noise (all resampled bits become zero).

LDA/STA writes (the normal 6502 store instructions) are always exact—the noise applies only to BRK noisy copies.

Board hyperparameters

The controller accepts a `boardParams` dictionary with scalar parameters and a `brkOps` registry that describes every BRK operand the board supports.

Scalar parameters.

Parameter	Default	Description
`pBitNoise`	1/2048	Per-bit resampling probability on BRK noisy copy (ε)
`pBitNoiseZero`	0.5	P(resampled bit = 0); 0.5 = fair coin, 1 = erasure (q)
`nSwapCycles`	0	Minimum remaining scheduler cycles for BRK copy/swap to succeed (0 = no limit)
`hasCompass`	false	Write orientation to $FA (shifted left 2 bits)

BRK operand registry (`brkOps`). The `brkOps` dictionary maps operation names to descriptors. Each descriptor specifies the operand range and whether the operation is enabled on this board. The code-side registry adds non-serialized metadata: cycle costs, address encoding, and handler functions.

Name	Range	Enabled	Cycles	Addr. encoding	Description
`reset`	$[0, 0]$	true	12	none	Reset PC to 0, yield
`swap`	$[1, 48]$	true	49	operand	Swap cell 0 with cell b ($O(1)$)
`copy`	$[49, 96]$	true	14,400	operand-48	Noisy copy to cell $b - 48$ ($O(M)$)
`sync`	$[97, 97]$	false	24	X,Y registers	Sync interrupt request
`async`	$[98, 98]$	false	24	X,Y registers	Async interrupt request

Operands 99–255 are unassigned by default and yield with no effect. Board owners can extend the registry by adding entries in this range, registering a handler function and cycle cost in the code-side `BRK_OP_REGISTRY`. Programs that issue an unrecognized operand simply yield; programs that use a new operand get the new capability on boards that support it and degrade gracefully on boards that don't.

At construction time, the controller builds a 256-entry dispatch table from `brkOps`: each entry is either the handler for the operation covering that operand, or null (yield). Dispatch is a single array lookup—no range comparisons at runtime.

For serialization, the state includes both `brkOps` and legacy `implementsMove/implementsCopy/impl` boolean flags (derived from `brkOps`) for backward compatibility with older readers. When loading state, `brkOps` takes precedence; if absent, the legacy flags are used to synthesize it.

The guiding principle when considering adding more software interrupts, or expanding the OS in any way, should be to not add any functionality that can't be justified as "physics".

  - After any copy or swap, control returns to the scheduler.

- A software interrupt due to a bad instruction is handled like BRK 0 (PC resets to 0, control returns to scheduler).

3 Randomness

Several aspects of the VM rely on randomness: the scheduling order and orientation of cells, the inter-interrupt timing, the four pseudorandom bytes written to zero-page addresses 0xFC–0xFF, and the bit-noise in the noisy copy operation.

In the reference implementation, all randomness is generated by a single deterministic pseudorandom number generator (a Mersenne Twister seeded at initialization). This makes simulations fully reproducible given the same seed, which is valuable for debugging, regression testing, and scientific reproducibility.

However, nothing in the VM specification requires deterministic randomness. Hardware or software implementations may substitute true random number generators (e.g. hardware entropy sources) for any or all of these random processes. The system's behavior should be qualitatively the same: the "physics" of the

board—scheduling fairness, copy noise, and the statistical properties of the RNG bytes visible to programs—depend only on the *distribution* of the random variables, not on their deterministic reproducibility.

Programs should not rely on correlations between successive RNG values or between the RNG and the scheduling order. A correct program must work under any source of randomness that satisfies the specified distributions.

4 Visualization Conventions

The following conventions describe how a cell may expose information to external viewers (debuggers, phone UIs, web dashboards). **These are conventions, not part of the VM specification.** The VM does not read or interpret these bytes; programs are free to use them for code or data. However, viewers and tooling may render cells based on this layout, so programs that follow the convention get richer visualization for free.

Offset	Size	Convention
0x3C0–0x3DF	32 bytes	16×16 monochrome bitmap (1 bit/pixel, MSB first)
0x3E0–0x3FB	28 bytes	ASCII display name
0x3FC–0x3FE	3 bytes	Reserved
0x3FF	1 byte	Hue (0–255 maps to 0–360° HSV)

Hue byte. The byte at 0x3FF sets the cell's color. The value 0–255 is interpreted as an HSV hue (0–360°). A nonzero hue is rendered at full saturation with brightness scaled by recent activity. Programs that write a hue byte get colored visualization for free; a zero hue falls back to the default activity-based heat coloring.

Monochrome bitmap. 32 bytes at 0x3C0–0x3DF encode a 16×16 pixel bitmap (1 bit per pixel, MSB first, row-major). This is available for detail views or cell inspectors. The hue byte provides the color; the bitmap provides the shape.

Display name. The 28 bytes at 0x3E0–0x3FB are interpreted as an ASCII string. The reference web app parses this as `[cssColor]:[iconifyIconName]` (e.g. `orange:bee`, `red:sword`). If no colon is present, the string is treated as an Iconify icon name from the `game-icons` set.

For SokoScript and other high-level languages: these conventions provide a natural way to expose cell type and state visually. A compiled SokoScript program could write its type name to 0x3E0, set a hue byte, and render its state as a bitmap, giving viewers a rich display without modifying the VM.

For the phone game: the PWA coin miner uses the hue byte for cell color when present, falling back to the overview color (computed from write/move recency via HSV mapping). The monochrome bitmap is available for future detail views.

5 Undocumented NMOS 6502 Opcodes

The NMOS 6502 has 105 opcodes not defined in the official instruction set. In 6502life, all 256 opcodes have defined behavior, divided into three categories:

5.1 Stable opcodes (faithfully emulated)

These combine documented operations in predictable ways and are well-understood from decades of reverse engineering:

- **LAX** (LDA + LDX): loads both A and X from memory.
- **SAX**: stores A & X to memory.
- **DCP** (DEC + CMP): decrements memory, then compares result with A.
- **ISC/ISB** (INC + SBC): increments memory, then subtracts from A.
- **SLO** (ASL + ORA): shifts left, then ORs with A.
- **RLA** (ROL + AND): rotates left, then ANDs with A.
- **SRE** (LSR + EOR): shifts right, then XORs with A.
- **RRA** (ROR + ADC): rotates right, then adds to A.
- **ANC**: AND immediate, copies bit 7 to carry.
- **ALR**: AND immediate, then LSR accumulator.
- **ARR**: AND immediate, then ROR with special flag handling.
- **AXS/SBX**: (A & X) − immediate → X.

These use the same addressing modes and cycle counts as their documented equivalents. Programs that exploit undocumented opcodes (common in C64 demo scenes) will execute correctly.

5.2 JAM/HLT opcodes (halt the cell)

Opcodes $02, $12, $22, $32, $42, $52, $62, $72, $92, $B2, $D2, $F2 freeze the CPU on real hardware. In 6502life, they set a per-cell "halted" flag. A halted cell does not execute until its memory is overwritten by a copy operation. This provides a natural "death" mechanism: random memory that decodes as a JAM instruction permanently stops that cell, creating selective pressure for programs that avoid JAM bytes.

5.3 Unstable opcodes (NOP with correct timing)

These depend on analog bus state and vary between physical chips. In 6502life they are treated as NOPs that consume the correct number of bytes and cycles but have no other effect. This includes XAA/ANE ($8B), AHX/SHA ($93, $9F), TAS/SHS ($9B), SHY ($9C), SHX ($9E), LAS ($BB), and various undocumented NOP variants with 1–3 byte, 2–5 cycle timing.

6 Terminology

The following terms are used consistently throughout the specification, implementation, and IDE documentation. Bold terms are normative.

Term	Definition
Board Storage	The persistent $B \times B \times M$ byte array holding all cell data—analogous to peeking at a hard drive. Running programs cannot see Board Storage directly; they see the Memory Map instead. The IDE Board Pane shows Board Storage in fixed, untranslated/unrotated coordinates.
Stored Cell	A contiguous 1024-byte (M-byte) block within Board Storage, addressed by grid coordinates (i, j).
Scheduler	The (pseudo)code that picks cells for execution, samples orientations and timer durations, and controls the running of 6502 code. Never visible to the programs themselves.
Active Cell	The Stored Cell currently scheduled for execution (coordinates $i_{\text{orig}}, j_{\text{orig}}$).
Active Frame	The Active Cell together with its sampled orientation. The Active Frame determines how the 7×7 neighborhood is mapped into the Memory Map.
Next IRQ Time	The cycle count at which the next timer interrupt will fire. Visible only to the Scheduler, not to programs.
Memory Map	The 49-cell (N^2-cell) RAM address space that the running program actually sees. Board Storage cells are loaded into the Memory Map in spiral order around the Active Cell, rotated by the Active Frame's orientation.
Currently Mapped Cells	The 49 Stored Cells that have been loaded into the Memory Map for the current Active Frame.
Register State	The 6502 CPU registers (A, X, Y, S, P, PC) during execution.
Stored Register State	The zero-page locations ($F9–$FF) in a Stored Cell (or its Memory Map image) that preserve the Register State between scheduling quanta.
Active Context	The upcoming bytes of the Memory Map in the 6502's execution pipeline—the instruction stream starting at the current PC.
Cursor Cell	(IDE) The Stored Cell that owns the memory address corresponding to the current IDE Map Cursor position.

IDE pane mapping:

- The **IDE Map Pane** shows Board Storage restricted to the Currently Mapped Cells. Because the mapping is known, the **IDE Map Cursor** always corresponds to a unique address in the Memory Map, displayed on the pane boundary.
- The **IDE Disassembler Pane** shows the Active Context by default, but can be pointed at any address in the Memory Map.
- The **IDE Board Pane** (minimap) shows a fixed, untranslated/unrotated

view of Board Storage. The Cursor Cell and the Active Cell are highlighted.

7 Shadow OS

The scheduler, interrupt handling, and BRK operations described above are implemented in the reference software as fast-path native code (JavaScript or Rust). However, all of these operations *could* be implemented as a 6502 program running on the same emulated CPU, given a small amount of dedicated hardware. We call this notional 6502 program the **Shadow OS**.

The Shadow OS is invisible to user programs. It occupies a separate address space that is mapped in only during interrupt servicing and BRK dispatch. Programs cannot read, write, or detect the Shadow OS; they see only the neighborhood RAM ($0000–$BFFF) and the geometric lookup tables ($E000–$EEFF). The only interface between programs and the OS is the BRK operand byte.

A reference 6502 implementation of the Shadow OS is provided in `shadow-os/shadow-os.asm`. It is not used by the JS or Rust reference implementations (which fast-path all BRK operations as native code), but serves to verify the cycle costs and prove that the OS is implementable on a real 6502 with the described hardware.

7.1 Shadow OS contract

The Shadow OS has a minimal and precisely bounded contract with user programs:

1. **Transparent local topology.** The OS implements the 7×7 neighborhood seamlessly via the memory map. Each scheduling quantum, it maps 49 cells of Board Storage into the program's address space ($0000–$BFFF), with a random origin and random rotation. The program sees a contiguous, coherent address space; it does not need to know that the underlying storage is a toroidal grid, or that the mapping changes each interrupt.

2. **Minimal geometry API.** The only "help" the OS provides is a set of read-only lookup tables at $E000–$EEFF. These tables encode the local symmetry group of the neighborhood: vector addition (translations), rotations (90°, 180°, 270°), reflections, and coordinate lookups (cell index $\leftrightarrow (x, y)$). These are purely geometric—they describe the group-like structure of the local topology and nothing else. They help programs reason about *where* things are, not *what* to do about them.

3. **No computational or strategic assistance.** The OS offers no arithmetic helpers, no pattern matching, no communication primitives, no memory management. It does not interpret cell contents or favor any program over another. All intelligence must emerge from the programs themselves.

4. **Opaque BRK interface.** Programs interact with the OS exclusively through BRK operands. They set registers and issue BRK; the OS performs the requested operation (move, copy, timer setup) and yields. Programs cannot observe how the operation is performed, only its effects.

7.2 Shadow OS memory map

During Shadow OS execution, the 6502 address space is remapped:

Address range	Contents
$0000–$03FF	Current cell (cell 0), read/write
$0400–$04FF	OS scratch page (256 bytes, private)
$C000–$C3FF	Shadow window: target cell, selected by `SHADOW_BANK`
$D000–$D003	Hardware registers (see below)
$E000–$EEFF	Geometric lookup tables (shared with user programs)
$EF00–$FDFF	Shadow OS code (ROM)
$FE00–$FFFF	Vectors and boot code (ROM)

During user program execution, the $C000–$FFFF region is unmapped (returns open-bus values or zero). Programs never see the Shadow OS code, the shadow window, or the hardware registers.

7.3 Shadow hardware registers

The Shadow OS requires four memory-mapped hardware registers, accessible only during OS execution:

Address	Name	Function
$D000	`SHADOW_BANK`	Write: maps neighborhood cell n into the shadow window at $C000–$C3FF.
$D001	`NOISE_BYTE`	Read: returns next byte from hardware LFSR, then clocks the shift register. Used for the RNG bytes at $FC–$FF. (The noisy copy uses the hardware noise gate on the shadow window write path, not this register.)
$D002	`TIMER_LO`	Read/write: interrupt countdown timer, low byte.
$D003	`TIMER_HI`	Read/write: interrupt countdown timer, high byte.

The shadow window at $C000–$C3FF is a second 1 KB bank selected by writing a cell index (0–48) to `SHADOW_BANK`. This is the only piece of address bus multiplexing beyond the neighborhood mapper that already exists for user programs. The necessity of this second bus—giving the Shadow OS simultaneous access to cell 0 (at $0000) and one partner cell (at $C000)—is as good an argument as any for restricting fast memory operations (swap and noisy copy) to only ever involve the active cell and one other partner. A hypothetical three-way swap would require a *third* shadow window, further complicating the address bus, with diminishing returns.

7.4 Shadow OS routines

Timer interrupt (NMI). The timer fires an NMI. The Shadow OS:

1. Saves the user's registers (A, X, Y, S, P, PC) to the cell's register save area ($F9–$FF). Clears the B flag. (∼67 cycles.)
2. If I is clear, writes back all 49 cells from working RAM to storage using exact copies through the shadow window ($49 \times M$ bytes). If I is set, skips writeback entirely (atomic abort, $O(1)$).
3. Writes four pseudorandom bytes from `NOISE_BYTE` to $FC–$FF. (∼28 cycles.)
4. Advances the scheduler: selects the next cell, samples a new orientation and timer duration.
5. Reads the new cell's 49-cell neighborhood from storage into working RAM using exact copies through the shadow window ($49 \times M$ bytes).
6. Restores the new cell's registers and executes RTI. (∼38 cycles.)

The register save/restore and RNG cost ∼133 cycles. The bulk of the context switch cost is the read-in and writeback copies, which use the same shadow-window hardware path as BRK noisy copy (but with the noise gate disabled). A DMA controller can accelerate these transfers.

BRK dispatch. On BRK, the OS extracts the operand byte (∼41 cycles of stack manipulation, which could be reduced by a hardware operand latch) and dispatches:

- **Operand 0** (reset): set PC to 0, jump to scheduler. Cost: ∼11 cycles.
- **Operands 1–48** (swap): look up the target's board cell index from a precomputed neighbor table, then exchange the page-table entries for cell 0 and the target cell. This is an $O(1)$ operation regardless of cell size—no data is copied. Set the B flag in the parent's saved P. Yield to scheduler. Cost: ∼49 cycles (12 for neighbor lookup, 26 for page-table swap, 11 for B flag and yield).
- **Operands 49–96** (noisy copy): look up the target cell and write its board index to `SHADOW_BANK`, mapping it into the shadow window at $C000. Copy all $M = 1024$ bytes from cell 0 ($0000–$03FF) to the shadow window ($C000–$C3FF). The hardware noise gate on the shadow window's write path automatically applies the per-bit noise model—the CPU performs a plain `STA`, and the hardware XORs each bit with the LFSR output at probability ε. This is an $O(M)$ operation: every byte must pass through the noise gate. Set the B flag. Yield to scheduler. Cost: ∼14,400 cycles (∼14 per byte: `LDA abs,Y` + `STA abs,Y` + `INY` + `BNE` $= 4+5+2+3 = 14$ cycles, times 1024 bytes, plus setup).

- **Operand 97** (sync interrupt): read X and Y from the user's saved registers, write to the timer hardware. Cost: ∼24 cycles.
- **Operand 98** (async interrupt): read X and Y, write delay to the timer hardware. Cost: ∼24 cycles.

The key asymmetry: swap is $O(1)$ because paged storage allows remapping without data movement, while noisy copy is inherently $O(M)$ because every byte must pass through the noise gate. Swap costs ∼49 cycles versus ∼14,400 for copy—a ratio of roughly 300:1, reflecting the fundamental difference between pointer manipulation and bulk data transfer.

For `nSwapCycles` gating, the reference implementation applies a single threshold for both operations. An alternative would be separate thresholds (e.g. ∼100 for swap, ∼15,000 for copy), but the single-threshold approach keeps the hyperparameter space small.

8 Hardware implementations

The VM can be implemented in hardware at various scales. We sketch three approaches: a minimal 6502-based board, and ARM-based emulator boards at three board sizes.

8.1 Minimal 6502 hardware

A physical 6502 running the Shadow OS requires the following custom hardware beyond a stock CPU:

1. **Paged cell storage.** Each cell's 1 KB occupies a page in external storage. A page table maps logical cell indices to physical page addresses. Swap is implemented by exchanging page-table entries. For a 16×16 board: 256 cells $\times$ 1 KB = 256 KB SRAM, plus a 256-entry page table (256 bytes if entries are single-byte page numbers).

2. **Neighborhood mapper.** On each scheduling event, the mapper configures 49 bank-select registers to present the new cell's 7×7 neighborhood in the 6502's \$0000–\$BFFF address space, applying the random rotation. With paged storage, this is $O(N^2)$ pointer setups (∼49 register writes), not $O(N^2M)$ byte copies—the data stays in place in physical storage and only the address translation changes. This is the most complex piece of custom logic: a combination of address translation ROM and bank-select latches.

   Read/write and undo. The address space at \$0000–\$BFFF is fast working SRAM, not directly mapped to storage. The Shadow OS explicitly copies data between working RAM and paged storage using exact (noiseless) copies through the shadow window—the same hardware path as the noisy copy, but with the noise gate disabled ($\varepsilon = 0$).

Before execution: the Shadow OS copies the 49 neighborhood cells from storage into working RAM (the "read"). With paged storage, this is 49 sequential shadow-window copies; a DMA controller can overlap the transfer with other work.

On commit (BRK, or timer with I clear): the Shadow OS copies all 49 cells from working RAM back to storage ($49 \times M$ bytes through the shadow window). This is unconditional—no dirty tracking is needed.

On undo (timer with I set): the Shadow OS simply skips the writeback. The storage retains its pre-quantum state. This is $O(1)$.

This is a realistic mechanism that uses no special write-tracking hardware—just the Shadow OS's own copy primitives and the shadow window used for BRK operations. The reference JS implementation models undo with an `undoHistory` hash map (tracking individual bytes for efficiency), but the semantics are identical.

3. **Shadow window.** A second 1 KB bank at $C000, selected by writing to the `SHADOW_BANK` register. This is a multiplexer on the address bus: when A15:A14 = 11 and A11:A10 = 00, route the low 10 address bits through the page table using the `SHADOW_BANK` index. This second shadow bus complicates the scheduler's address bus multiplexing, but only modestly—it reuses the same page table and storage bus, adding one extra latch and mux stage.

4. **Noise LFSR and noise gate.** A free-running 16-bit Galois LFSR (two 74HC flipflop chips plus an XOR gate, ∼3 TTL packages). Reading $D001 latches and returns the low 8 bits, then clocks the register.

   The noise gate sits on the data bus between the CPU and the shadow window's backing storage. On any write to $C000–$C3FF, the gate applies per-bit resampling: each bit of the written byte independently passes through or is replaced by an LFSR bit, controlled by a comparator against the ε threshold. With $\varepsilon = 0$, all bits pass through (perfect copy). This adds a single XOR gate and comparator latch to the write path (∼2 extra TTL packages) but makes noisy copy transparent to the CPU—the Shadow OS executes a plain `STA` and the hardware applies the noise.

5. **Interrupt timer.** A 16-bit countdown register at $D002–$D003, triggering NMI on underflow. One 74HC4520 dual counter or equivalent.

Total additional hardware: ∼10–15 TTL/CMOS chips beyond the 6502, ROM, and SRAM. Feasible as a wire-wrap prototype or a small PCB.

8.2 ARM emulator boards

An ARM processor running a 6502 emulator replaces all custom hardware with software. The "Shadow OS" becomes native ARM code in the BRK handler—swap and copy are direct memory operations on the DRAM cell array, and the cycle counter is advanced by the Shadow OS cost to maintain timing fidelity.

Emulator architecture. The ARM stores all cell data in DRAM. For each scheduling quantum: (1) set up an address-translation table mapping 6502 addresses to DRAM pointers (49 entries, one per neighborhood cell, pre-rotated); (2) run the 6502 emulator for N cycles, translating each memory access through the table; (3) on BRK or timer interrupt, execute the operation as native ARM code (e.g. a 1 KB `memcpy` for copy, or a page-table swap for move); (4) advance the emulated cycle counter by the Shadow OS cost.

Each 6502 instruction costs ∼15–25 ARM instructions (address translation + opcode dispatch + execution). At 8 MHz, this yields ∼400k emulated 6502 cycles per second.

Scaling.

Board	Cells	RAM	ARM chip	Cell visits/sec
16 × 16	256	256 KB	ARM2 @ 8 MHz (1987)	∼8/cell/s
32 × 32	1024	1 MB	ARM2 @ 8 MHz (1988)	∼2/cell/s
64 × 64	4096	4 MB	ARM3 @ 25 MHz (1989)	∼1.5/cell/s

The 16×16 board fits comfortably in an Acorn Archimedes A305 (512 KB RAM, 1987). Each cell is visited ∼8 times per second—fast enough for interactive visualization.

The 32 × 32 board requires 1 MB (Archimedes A440 or equivalent). Cells are visited ∼2 times per second. Evolution proceeds at one-quarter the rate of the 16 × 16 board in wall-clock time.

The 64 × 64 board needs 4 MB of DRAM, expensive but feasible by 1989. The ARM3 at 25 MHz recovers most of the per-cell throughput lost to the larger board.

In all cases, BRK swap and copy execute as native ARM code in microseconds, but the emulated cycle counter is advanced by the full Shadow OS cost (∼49 cycles for swap, ∼14,400 for noisy copy), preserving the timing semantics that programs experience.

9 Links

- Avida (Ofria and Wilke, 2004; Ortega et al., 2023) `https://en.wikipedia.org/wiki/Avida_(software)`
- Core War `https://en.wikipedia.org/wiki/Core_War`
- The BBC Micro Bot `https://mastodon.me.uk/@bbcmicrobot`

References

Ofria, C. and Wilke, C. O. (2004). Avida: a software platform for research in computational evolutionary biology. *Artif Life*, 10(2):191–229.

Ortega, R., Wulff, E., and Fortuna, M. A. (2023). Ontology for the Avida digital evolution platform. *Sci Data*, 10(1):608.